# Appendix A

## Exam Topics: Java SE 21 Developer Professional

Please note that all information presented in this appendix is valid as of November, 2024. It is imperative that readers visit the exam website mentioned in this appendix regularly while preparing for the exam, as Oracle is known to change the exam objectives intermittently.

| | |
|---|---|
| Exam Name: *Java SE 21 Developer Professional* | Duration: *120 minutes* |
| Exam Number: *1Z0-830* | Number of Questions: *50* |
| Certification: *Oracle Certified Professional:* | Passing Score: 68% |
| *Java SE 21 Developer* | Exam Format: *Multiple Choice* |
| | Exam Price: *$245* |

Candidates must pass the *Java SE 21 Developer Professional Exam* in order to qualify as an *Oracle Certified Professional: Java SE 21 Developer*. Pertinent information about this exam can be found at:

**Click here to view code image**

```
https://education.oracle.com/java-se-21-developer-professional/pexam_1Z0-830
```

This web page also lists the exam topics defined by Oracle. The topics are organized in *sections*, and each section is *reproduced verbatim* in the table at the end of this appendix. For each section, we have provided references to where the exam topics (we call them *objectives*) in the section are covered in this guide. The prefixes **Part I** and **Part II** are used in the references to indicate whether a topic is covered in **Part I**: **OCP Java SE 21 Developer** or **Part II**: **OCP Java SE 17 Developer** of this guide, respectively.

**Part I** and **Part II** together provide a comprehensive coverage for all the topics on the *Java SE 21 Developer Professional Exam*.

General information about taking the exam can be found at the exam site. Oracle has also specified certain important assumptions about the exam questions, which can also be found at the exam site. In addition to the exam objectives and the assumptions, the exam web page lists a number of topics that the candidate is expected to know, but the exam will not include any direct questions on these topics. We have classified them as *supplementary topics* (**p. 111**).

| | | |
|---|---|---|
| **Section 1: Handling Date, Time, Text, Numeric and Boolean Values** | | |
| [1.1] | Use primitives and wrapper classes. Evaluate arithmetic and boolean expressions, using the Math API and by applying precedence rules, type conversions, and casting. | **Part II**: §2.2, p. 41 *to* **Part II**: §2.19, p. 92 **Part II**: §8.3, p. 431 **Part II**: §8.6, p. 480 |
| [1.2] | Manipulate text, including text blocks, using String and StringBuilder classes. | **Part II**: §8.4, p. 441 **Part II**: §8.5, p. 466 **Part I**: §C.1, p. 121 |
| [1.3] | Manipulate date, time, duration, period, instant and time-zone objects including daylight saving time using Date-Time API. | **Part II**: §17.1, p. 1028 *to* **Part II**: §17.7, p. 1076 |
| **Section 2: Controlling Program Flow** | | |

| Section 1: Handling Date, Time, Text, Numeric and Boolean Values | | |
|---|---|---|
| [2.1] | Create program flow control constructs including if/else, switch statements and expressions, loops, and break and continue statements. | **Part II**: §4.1, p. 154 *to* **Part II**: §4.13, p. 186 **Part I**: Chapter 1, p.1 |
| **Section 3: Using Object-Oriented Concepts in Java** | | |
| [3.1] | Declare and instantiate Java objects, including nested class objects, and explain the object life-cycle including creation, reassigning references, and garbage collection. | **Part II**: §9.1, p. 493 *to* **Part II**: §9.6, p. 523 **Part II**: §10.1, p. 535 *to* **Part II**: §10.4, p. 539 |
| [3.2] | Create classes and records, and define and use instance and static fields and methods, constructors, and instance and static initializers. | **Part II**: §3.1, p. 99 *to* **Part II**: §3.8, p. 112 **Part II**: §5.14, p. 301 **Part II**: §10.5, p. 542 |

| Section 1: Handling Date, Time, Text, Numeric and Boolean Values | | |
|---|---|---|
| | | *to* **Part II**: §10.9, p. 557 |
| [3.3] | Implement overloaded methods, including var-arg methods | **Part II**: §3.6, p. 108 **Part II**: §3.11, p. 136 **Part II**: §5.1, p. 204 |
| [3.4] | Understand variable scopes, apply encapsulation, and create immutable objects. Use local variable type inference. | **Part II**: §3.13, p. 143 **Part II**: §6.1, p. 326 **Part II**: §6.6, p. 354 **Part II**: §6.7, p. 358 |
| [3.5] | Implement inheritance, including abstract and sealed types as well as record classes. Override methods, including that of the Object class. Implement polymorphism and differentiate between object type and reference type. Perform reference type casting, identify object types using the instanceof operator, and pattern matching with the instanceof operator and the switch construct. | **Part II**: §5.1, p. 193 *to* **Part II**: §5.4, p. 220 **Part II**: §5.11, p. 271 **Part II**: §5.12, p. 280 **Part II**: §5.15, p. |

| | | |
|---|---|---|
| **Section 1: Handling Date, Time, Text, Numeric and Boolean Values** | | |
| | | **313**<br>**Part II**: **§14.1**, **p. 745**<br>*to*<br>**Part II**: **§14.3**, **p. 755**<br>**Part I**: **Chapter 1**, **p.1** |
| [3.6] | Create and use interfaces, identify functional interfaces, and utilize private, static, and default interface methods. | **Part II**: **§5.6**, **p. 239** |
| [3.7] | Create and use enum types with fields, methods, and constructors. | **Part II**: **§5.13**, **p. 289** |
| **Section 4: Handling Exceptions** | | |
| [4.1] | Handle exceptions using try/catch/finally, try-with-resources, and multi-catch blocks, including custom exceptions. | **Part II**: **§7.2**, **p. 377**<br>**Part II**: **§7.3**, **p. 377**<br>**Part II**: **§7.6**, **p. 399**<br>**Part II**: **§7.7**, **p. 409** |
| **Section 5: Working with Arrays and Collections** | | |
| [5.1] | Create arrays, List, Set, Map and Deque collections, and add, remove, update, retrieve and sort their elements. | **Part II**: **§3.9**, **p. 118**<br>**Part II**: **§11.1**, **p. 567** |

| Section 1: Handling Date, Time, Text, Numeric and Boolean Values | | |
|---|---|---|
| | | *to* **Part II**: **§11.13**, **p. 625** **Part II**: **§12.1**, **p. 646** *to* **Part II**: **§12.8**, **p. 664** **Part II**: **§14.4**, **p. 763** **Part II**: **§14.5**, **p. 771** **Part II**: **§15.1**, **p. 785** *to* **Part II**: **§15.12**, **p. 866** **Part I**: **Chapter 2**, **p.45** |
| **Section 6: Working with Streams and Lambda Expressions** | | |
| [6.1] | Use Java object and primitive Streams, including lambda expressions implementing functional interfaces, to create, filter, transform, process, and sort data. | **Part II**: **§13.1**, **p. 677** *to* **Part II**: **§13.14**, **p.** |

| | | |
|---|---|---|
| **Section 1: Handling Date, Time, Text, Numeric and Boolean Values** | | |
| | | 735<br>**Part II**: §16.3, p. 886<br>*to*<br>**Part II**: §16.7, p. 948 |
| [6.2] | Perform decomposition, concatenation and reduction, and grouping and partitioning on sequential and parallel streams. | **Part II**: §16.7, p. 948<br>**Part II**: §16.8, p. 980<br>**Part II**: §16.9, p. 1011 |
| **Section 7: Packaging and Deploying Java Code** | | |
| [7.1] | Define modules and expose module content including that by reflection, and declare module dependencies, define services, providers, and consumers. | **Part II**: §19.1, p. 1167<br>*to*<br>**Part II**: §19.5, p. 1183<br>**Part II**: §19.8, p. 1195<br>**Part II**: §19.9, p. 1200 |

| Section 1: Handling Date, Time, Text, Numeric and Boolean Values | | |
|---|---|---|
| [7.2] | Compile Java code, create modular and non-modular jars, runtime images, and implement migration to modules using unnamed and automatic modules. | **Part II**: **§19.6**, **p. 1190** **Part II**: **§19.7**, **p. 1193** **Part II**: **§19.10**, **p. 1208** *to* **Part II**: **§19.14**, **p. 1222** |
| **Section 8: Managing Concurrent Code Execution** | | |
| [8.1] | Create both platform and virtual threads. Use both Runnable and Callable objects, manage the thread lifecycle, and use different Executor services and concurrent API to run tasks. | **Part II**: **§22.1**, **p. 1371** *to* **Part II**: **§22.5**, **p. 1412** **Part II**: **§23.2**, **p. 1427** **Part I**: **Chapter 3**, **p.73** |
| [8.2] | Develop thread-safe code, using different locking mechanisms and concurrent API. | **Part II**: **§23.4**, **p. 1455** *to* **Part II**: |

| | | |
|---|---|---|
| **Section 1: Handling Date, Time, Text, Numeric and Boolean Values** | | |
| | | §23.7, p. 1486 |
| [8.3] | Process Java collections concurrently and utilize parallel streams. | Part II: §16.9, p. 1011<br>Part II: §23.7, p. 1486 |
| **Section 9: Using Java I/O API** | | |
| [9.1] | Read and write console and file data using I/O streams. | Part II: §20.1, p. 1237<br>*to*<br>Part II: §20.4, p. 1260 |
| [9.2] | Serialize and de-serialize Java objects. | Part II: §20.5, p. 1265 |
| [9.3] | Construct, traverse, create, read, and write Path objects and their properties using the java.nio.file API. | Part II: §21.1, p. 1291<br>*to*<br>Part II: §21.8, p. 1349 |
| **Section 10: Implementing Localization** | | |

| Section 1: Handling Date, Time, Text, Numeric and Boolean Values | | |
|---|---|---|
| [10.1] | Implement localization using locales and resource bundles. Parse and format messages, dates, times, and numbers, including currency and percentage values. | **Part II**: **§18.1**, **p. 1100** *to* **Part II**: **§18.7**, **p. 1143** **Part I**: **§C.2**, **p. 125** |
| **Section 11: Supplementary Topics** | | |
| [11.1] | Understand the basics of Java Logging API. | **Part II**: **Appendix G**, **p.1753** |
| [11.2] | Use Annotations such as Override, FunctionalInterface, Deprecated, SuppressWarnings, and SafeVarargs. | **Part II**: **§25.5**, **p. 1581** |
| [11.3] | Use generics, including wildcards. | **Part II**: **Chapter 11**, **p. 565** |

Previous chapter
3. Virtual Threads

Next chapter
Appendix B. Annotated Answers to Review Questions